LUT University
BK10A6400 Basics of FE Analysis
Marko Matikainen

Autumn 2026

Bonus Task 4 (draft – final version will be published soon)
Thick beam structure analyzed with a two-dimensional four-node bilinear solid (plane) element
Possibility to earn up to six bonus points based on a quiz.
The quiz will be held on 26th of October.
No report submission is required. Students are strongly recommended to complete the bonus task before the quiz, since the quiz questions will be based on the results and topics covered in this task.

Your task is to derive a two-dimensional quadrilateral four-node bilinear solid element (plane element "Q4") via MATLAB/Python symbolic toolbox, and use the derived element for solving displacements at the free end of a thick cantilever beam. Derive shape functions and find elemental stiffness matrix via strain energy based on a plane stress assumption. Code your FE-solution in MATLAB/Python using meshes 1×1, 2×1 and $nx \times ny$ where $nx \times ny$ is the fine mesh for the converged solution. Use an equally distributed force at the free end of the beam. Solve nodal displacements at a free end and compare results to the analytical solution.

To solve the problem, use the given Matlab/Python templates:

run_DeriveQ4_template_v11.(m/py)	run firstly to export shape functions and stiffness matrix
run_plotShapef_template_v11.(m/py)	Approximate displacement field with given nodal displacements.
run_LinStatic_Q4_BonusTask5_template_v11.(m/py)	Solves cantilever problem
ElemDOFsConnectivity_Q4.(m/py)	Makes DOF's connectivity from nodal connectivity

Firstly, modify the `run_DeriveQ4_template_v11.m` (for MATLAB) or `run_DeriveQ4_template_v11.py` (for Python) file to derive the shape functions and the stiffness matrix.

These scripts generate:

- the **shape functions** in `ShapeFunctionsQ4.m` (MATLAB) or `ShapeFunctionsQ4.py` (Python), and
- the **element stiffness matrix** in `KlocQ4.m` (MATLAB) or `KlocQ4.py` (Python).

The shape functions are used to visualise the displacement field and compute the strain energy in

- `run_plotShapef_template_v11.m` (MATLAB) or
- `run_plotShapef_template_v11.py` (Python)

The stiffness matrix is then used in `run_LinStatic_Q4_BonusTask5_template_v11.m` (MATLAB) or `run_LinStatic_Q4_BonusTask5_template_v11.py` (Python) to solve the displacement at the free end of a thick cantilever problem using plane elements with a 2×1 mesh.

The parameters of the problem are given in Table.

Force, F [N]	Young's mod., E [Pa]	Poisson's ratio, ν	Length, L [m]	Height, H [m]	Width, W [m]
62500	$2.07 \cdot 10^{11}$	0.3	2	Student-spesific	0.1

The height H of the beam depends on the last digit of your student number. Let $n_{\text{student}} \in 0, 1, \ldots, 9$ denote the last digit of your student number the height of the beam is defined as $H = 0.40 + 0.02, n_{\text{student}}$.

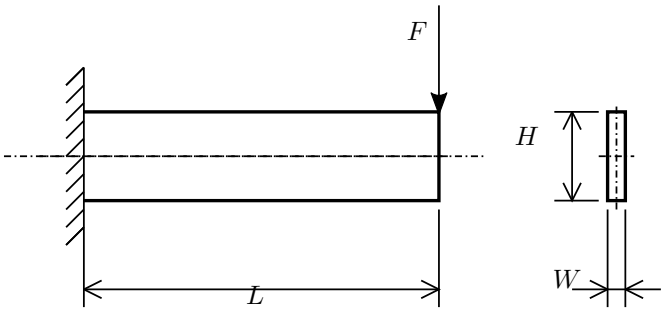


Figure 1. Thick cantilever problem

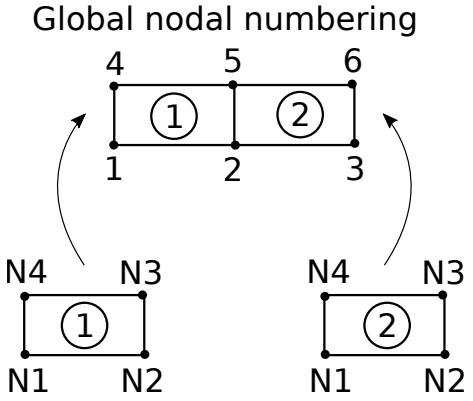


Figure 2. FE discretisation of the cantilever problem

Use the value of H corresponding to your own student number throughout the finite element analysis, mesh refinement study, and comparison with the analytical solution.

For the analytical solution, calculate the cross-sectional area and the second moment of area using

$$A = WH, \tag{1}$$

and

$$I = \frac{WH^3}{12}. \tag{2}$$

Please note that no points will be awarded without participation in the quiz. The quiz will be organized during the lecture. Remote participation is not possible.

Please note that verifying displacement field approximation and strain energy element's size L_x=2 m, L_y=2 m and L_z=0.1 m need to be used. Bonus points will be awarded as follows:

- Correct displacement field approximation with derived shape functions (0.5 point). Modify and run run_DeriveQ4_template_v10.m or run_DeriveQ4_template_v10.py file to export shape functions, internal forces and stiffness matrix. You can test your displacement field approximation and internal forces function by modifying the run_plotShapef_template_v11.m or run_plotShapef_template_v11.py file. Give a 0.01 m displacement in the vertical direction at element's local nodes 2 and 3, and approximate the displacement field $u_h(x = 0, y = 0) = \boldsymbol{N}\boldsymbol{u}$ and $v_h(x = 0, y = 0) = \boldsymbol{N}\boldsymbol{v}$. Use

element's size L_x=2 m, L_y=2 m and L_z=0.1 m in run_plotShapef_template_v11.m or run_plotShapef_template_v11.py file to check the displacement field.

- Correct strain energy (0.5 point). Finish strain energy form and export function for the strain energy with run_DeriveQ4_template_v10.m or run_DeriveQ4_template_v10.py file. Test the function providing a 0.01 m nodal displacement in the vertical direction at element's local nodes 2 and 3, and evaluate the vector of internal forces. Use element's size L_x=2 m, L_y=2 m and L_z=0.1 m in run_plotShapef_template_v10.m or run_plotShapef_template_v10.py file to check strain energy.
- Global assembling after the elimination of constraints (0.5 point). Compute the determinant and submit the result.
- Correct global nodal displacements $\{u_3, v_3, u_6, v_6\}$ at the free end with an element mesh 2×1. Be careful with signs (0.5 point).
- Find converged global nodal displacements at the free end with an element mesh $nx \times ny$. Now you end a very fine mesh and you will have many elements also in the thickness direction, so it makes sense to take the average of displacements in u_x and u_y direction. Be careful with signs (0.5 point).
- Solve maximum displacement based on Timoshenko's beam theory (1 point) and compare the result with FE-solution.

Tips

Analytical solution based on Timoshenko's beam theory (linear theory)

$$\frac{d^2v}{dx^2} = -\frac{M(x)}{EI(x)} - \frac{\kappa_{xy} q(x)}{GA} \tag{3}$$

Maximum deflection for the cantilever beam under applied force at the free end can be found as:

$$\delta_{max} = \underbrace{\frac{FL^3}{3EI}}_{\text{bending}} + \kappa_{xy} \underbrace{\frac{FL}{GA}}_{\text{shear}} \tag{4}$$

where κ_{xy} is a shear correction factor (needed to use in the beams based on the first-order shear deformation theory (FSDT)). It can be approximated to be $6/5$ for a rectangular cross-section.

Derivation of shape functions

Displacement field interpolations:

$$\begin{cases} u_h(x,y) := a_0 + a_1 x + a_2 y + a_3 xy \\ v_h(x,y) := b_0 + b_1 x + b_2 y + b_3 xy \end{cases}.$$

Displacement interpolation is the same for both displacements $a \Rightarrow$ same shape functions. Four unknowns, four eqs:

$$\begin{cases} u_h(-l_x/2, -l_y/2) := a_0 - \frac{1}{2} a_1 l_x - \frac{1}{2} a_2 l_y + \frac{1}{4} a_3 \, l_x \, l_y = u_1 \\ u_h(l_x/2, -l_y/2) := a_0 + \frac{1}{2} a_1 l_x - \frac{1}{2} a_2 l_y - \frac{1}{4} a_3 \, l_x \, l_y = u_2 \\ u_h(l_x/2, l_y/2) := a_0 + \frac{1}{2} a_1 l_x + \frac{1}{2} a_2 l_y + \frac{1}{4} a_3 \, l_x \, l_y = u_3 \\ u_h(-l_x/2, l_y/2) := a_0 - \frac{1}{2} a_1 l_x + \frac{1}{2} a_2 l_y - \frac{1}{4} a_3 \, l_x \, l_y = u_4 \end{cases}.$$

By solving a_i:s:

$$\boldsymbol{a} = \boldsymbol{A}^{-1}\boldsymbol{u}_u \tag{5}$$

and substituting them into the Eq.(6):

$$u_h = \boldsymbol{a}^T\boldsymbol{p} = \boldsymbol{A}^{-T}\boldsymbol{u}_u^T\boldsymbol{p}^T = \underbrace{\boldsymbol{A}^{-T}\boldsymbol{p}}_{\boldsymbol{N}}\boldsymbol{u}_u \tag{6}$$

where $\boldsymbol{N}$ includes shape functions.

Please take a look at more details in the lecture slides.

Derivation of stiffness matrix

$$W_{int} = \frac{1}{2}\int_V \boldsymbol{\sigma}\boldsymbol{\varepsilon}\,\mathrm{d}V = \frac{1}{2}\boldsymbol{u}^T \underbrace{l_z\int_x\int_y \boldsymbol{B}^T\boldsymbol{D}\boldsymbol{B}\,\mathrm{d}y\mathrm{d}x}_{\overline{\boldsymbol{K}}}\boldsymbol{u} \tag{7}$$

$$\overline{\boldsymbol{K}} = \int_V \boldsymbol{B}^T\boldsymbol{D}\boldsymbol{B}\,\mathrm{d}V = l_z\int_x\int_y \boldsymbol{B}^T\boldsymbol{D}\boldsymbol{B}\,\mathrm{d}y\mathrm{d}x = l_z\int_{-l_x}^{l_x}\int_{-l_y}^{l_y} \boldsymbol{B}^T\boldsymbol{D}\boldsymbol{B}\,\mathrm{d}y\mathrm{d}x \tag{8}$$

or more generally as:

$$\boldsymbol{f}_{int} = \frac{\partial W_{int}}{\partial \boldsymbol{u}} \quad ; \quad \overline{\boldsymbol{K}} = \frac{\partial \boldsymbol{f}_{int}}{\partial \boldsymbol{u}}$$

Assembling, BC's, applied forces and solution

You need to distribute force F for nodes at the free end. In static code, you need to uncomment lines and add forces with respect to correct DOF's.

```
%fglob(?)=?;        % add loading
%fglob(?)=?;        % add loading
```

Note that $\boldsymbol{K}_{G_{red}}$ is symmetric i.e. $k_{ij} = k_{ji}$ (if not, there is a typo) and $\det \boldsymbol{K}_{G_{red}} > 0$ (if not, check BC's).

And then we solve displacements (in a global coordinate system).

$$\boldsymbol{u}_{G_{red}} = \boldsymbol{K}_{G_{red}}^{-1}\boldsymbol{f}_{G_{red}}$$

In the case of a mesh of 2×1, you need to add one more element in the nodal connectivity matrix:

```
ElemNodeConnectivity=[1,2,4,3];             % for one element. You need two lines for ...
    two elements...
```

and then the element's DOF's connectivity is handled via ElemDOFsConnectivity_Q4.m function.

```
% Lets use functions that creates DOFs connectivity
ElemDOFsConnectivity=ElemDOFsConnectivity_Q4(ElemNodeConnectivity,NumberOfElems);
```

Note you need to change also lines

```
nxElems=1;                                  % This you need to change. Discretization ...
    in x-direction
NumberOfNodes=4;
```